

VIANNA JUNIOR
INSTITUTO

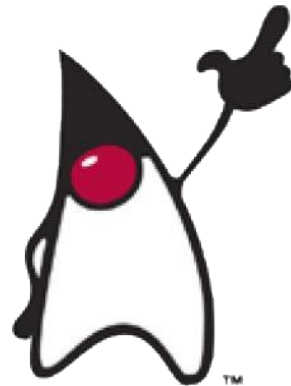


Teste de Software

Introdução



Apresentação, Conceitos



Professor Daves Martins

Doutorando em Eng. Elétrica

Mestre em Computação de Alto Desempenho pela UFRJ

Especialista em Banco de Dados

Analista Web – UFJF

Professor Efetivo – IFET JF

Sumário da Aula

Apresentação

Ementa

Instruções

Conceitos

Teste, Manutenção e Governança

Do que trata a disciplina?



- ✓ Dominar os princípios chaves e principais técnicas do teste de software
- ✓ Entregar uma “**solução**” produto/software para o cliente
- ✓ Com níveis de qualidade aceitáveis.
- ✓ Com facilidade de melhorias.
- ✓ Manter e gerir um processo de manutenção de Software
- ✓ Deploy e Devops

CLIENTE – TEM PROBLEMAS



Processo de Desenvolvimento de software:
Requisitos
Projeto
Código
Teste

Engenharia de Software Tradicional

CLIENTE – SOLUÇÃO



Apresentação



- Mas como funciona o desenvolvimento após a entrega para o cliente?
 - Qual Custo?
 - Como controlar o que fazer? Quem fez
 - E quando o software não tem um fim?
-
- Precisamos de um ambiente, “processo”, muito bem organizado e integrado.

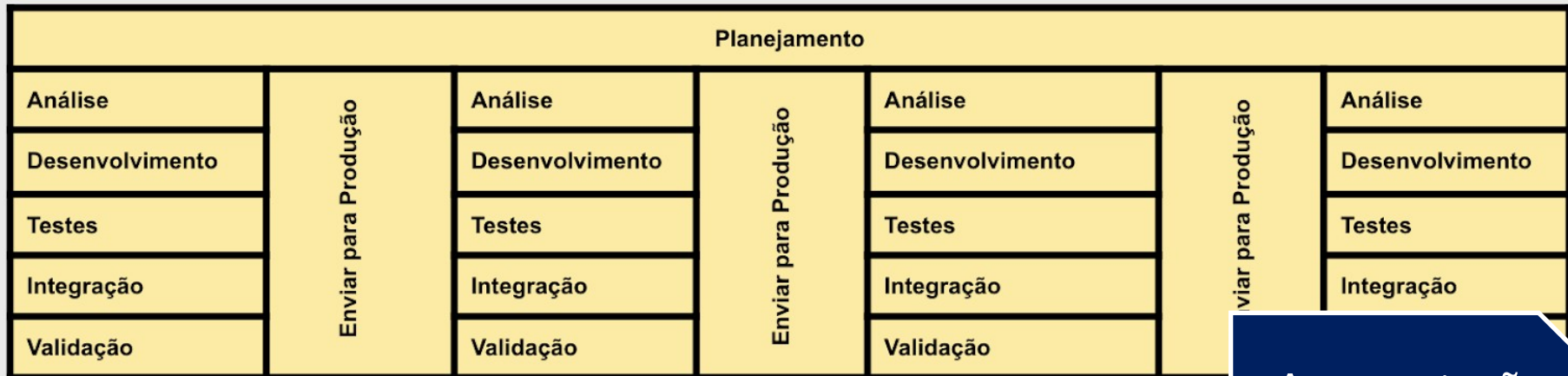
Metodologias ágeis e tradicionais no desenvolvimento de software

Metodologias Tradicionais



Desenvolvimento ágil

Tempo



Apresentação

- ✓ **Planejar e criar testes automatizados.**
- ✓ **Configurar e trabalhar em um ambiente de entrega de software continua e de qualidade.**
- ✓ **Pipeline (linha de produção de software)**
- ✓ **DEVOPS**

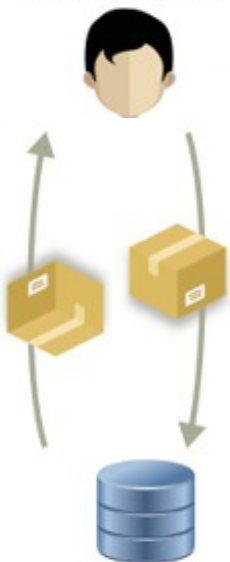
Controle de Mudança

Técnicas:
Kanban, Estórias, ...
Ferramentas:
Trac, redmine, ...



Desenvolvedor

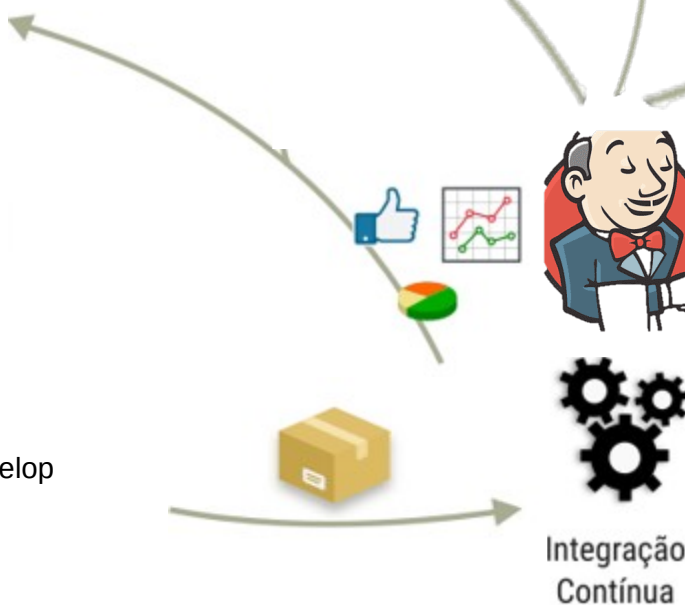
Técnicas:
Tdd, bdd, ...
Ferramentas:
NetBeans, Eclipse, ...
SmartGit,



Técnicas:
Gitflow, master/develop
Ferramentas:
github, gitlab ...

Controle de Versão

AWS, heroku, docker



Pipeline:
Compilar, testar, infra
Ferramentas:
Jenkins, Travis

Integração Contínua

DEVOPS

O que é o DevOps?

A palavra DevOps é uma contração de "Desenvolvimento" (Development) e "Operações" (Operations).



O DevOps é uma nova tendência na indústria de TI que visa melhorar a agilidade do Serviço de entrega de TI. O movimento enfatiza a comunicação transparente, a colaboração juntamente com a integração entre o software de Desenvolvedores e operações de TI.

DevOps reconhece que os desenvolvedores e operadores de TI não são grupos isolados que podem interagir uns com os outros, mas não realmente trabalhar juntos.

Os objetivos básicos do DevOps são estabelecer um ambiente em que códigos, testes e desenvolvimento de software possam ser feitos com rapidez, frequência e segurança.

Não há uma única ferramenta DevOps que trabalhe na colaboração e integração entre as equipes de desenvolvedores, testadores e operações.

Uma cadeia de ferramentas de DevOps é usada, e consiste em várias ferramentas que se ajustam em diversas categorias do processo em todas as fases, desde o desenvolvimento até a implementação.

Essas ferramentas são utilizadas nos processos que envolvem os times de codificação, construção, teste, embalagem, liberação, configuração e monitoramento.

Mas e a Governança de TI?

Sem controle, não pode haver qualquer tipo de gerenciamento.

“governança de TI é de responsabilidade dos executivos e da alta direção, consistindo em aspectos de liderança, estrutura organizacional e processos que garantam que a área de TI da organização suporte e aprimore os objetivos e as estratégias da organização.”

Ferramentas para as aulas

SmartGit

Bitvise SSH Client

NetBeans

Mysql Workbench

Criar conta no:

GitLab

No-ip

AWS – Mediante Convite do professor

- ✓ Introdução
- ✓ Conceitos
- ✓ Porque testar software?
- ✓ Fundamentos do teste de software
 - a. Níveis de Teste (quando testar?)
 - b. Técnicas de Teste (como testar?)
 - c. Tipos de Teste (o que testar?)

- ✓ Teste de Software no Processo de desenvolvimento
 - ✓ Modelo Tradicional
 - ✓ Modelo em V
 - ✓ Pirâmide de Teste
- ✓ Inspeção
- ✓ Documentação de Teste
 - ✓ Plano de teste
 - ✓ Caso de Teste

- ✓ Codificação e Teste de Software
 - ✓ Testes Automatizados x Testes Manuais
 - ✓ Testes unitários
 - ✓ Testes de integração
 - ✓ Testes funcionais
 - ✓ Teste de Cobertura
 - ✓ TDD

- ✓ Ambientes de Teste/Deploy
 - ✓ Homologação
 - ✓ QA
 - ✓ Produção
- ✓ Entrega Continua ou Deploy Contínuo.

- ✓ Manutenção de Software
 - ✓ Controle de tarefas
 - ✓ Versionamento Semântico
 - ✓ Métricas de Qualidade
 - ✓ Refatoração

- ✓ Deploy Contínuo
 - ✓ GitFlow
 - ✓ Pipeline
 - ✓ Devops

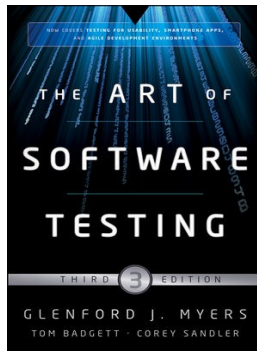
Considerações Importantes

- Frequência
- Calendário
- Trabalhos

Referências Bibliográficas



DELAMARO, M.E.; MALDONADO, J.C.; JINO, M.
Introdução ao Teste de Software. Elsevier Editora, 2007.



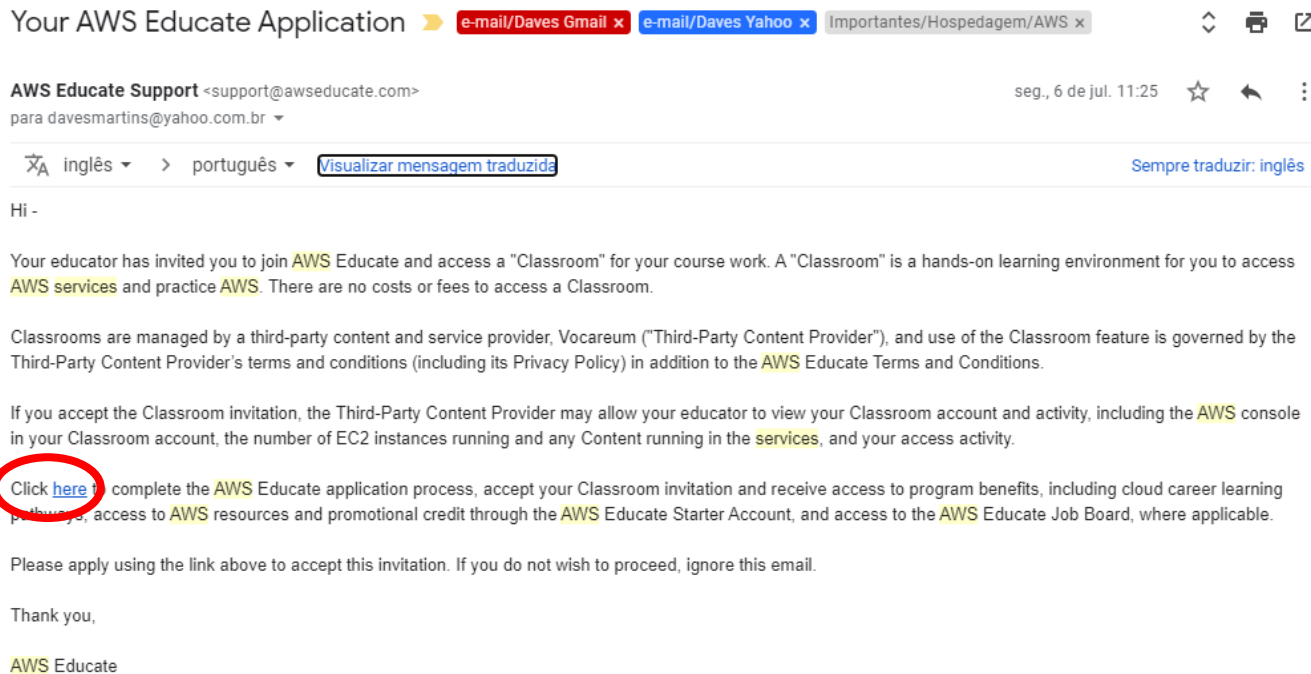
MYERS, G., BADGETT, T., THOMAS, T., SANDLER, C.,
The Art of Software Testing. Wiley; 3rd Edition, ISBN 978-
1118031964. 2011.

- ✓ Criar Conta GitLab
 - ✓ gitlab.com
 - ✓ Seguir o vídeo disponibilizado

- ✓ Criar Conta No-Ip (DNS Gratuíto)
 - ✓ www.noip.com
 - ✓ Criar 3 domínios:
 - ✓ qa-nome.ddns.net
 - ✓ homolog-nome.ddns.net
 - ✓ producao-nome.ddns.net
- ✓ Expiram em 30 dias, devem ser renovados.

- ✓ Criar Conta na AWS for Education (Servidores Gratuíto)
 - ✓ Terá um crédito de \$50,00
 - ✓ **Não pedirá cartão de crédito**
 - ✓ Todos irão receber no email do vianna um convite por email
 - ✓ Siga exatamente os passos a seguir:

- ✓ AWS – 1º Passo
 - ✓ Receberá uma email conforme abaixo
 - ✓ Clique e faça o cadastro



✓ AWS – 2º Passo

✓ Complete o cadastro



Apply to join AWS Educate

Step 2/3: Tell us about yourself

Preferred Language:

English ▼

School or Institution Name

Start typing the name of your school and select from the list. If you don't see your school, enter the full name, example: Harvard University

Country ▼

First Name

Last Name

your_name@gmail.com

Please provide a valid, current email issued by your institution. Example: your_name@your_school.edu

Graduation Month ▼

Graduation Year ▼

Birth Month ▼

Birth Year ▼

Promo Code (optional) ⓘ

[Frequently Asked Questions](#)

Please click the box below to help assure that a person and not an automated program is submitting this application. If a set of letters is displayed enter them on the line. If you have any difficulty with the letters, you can click the reload icon to get a new set of letters, or click the headphones to hear audio of what to enter.

- ✓ AWS – 3º Passo
 - ✓ Leia os termos até o final, e aceite



Inscriva-se para ingressar na AWS Educate

Termos e condições

Idioma preferencial:

English

TERMOS E CONDIÇÕES DO PROGRAMA DE ENSINO DA AWS

(Última atualização em 30 de abril de 2019)

1.0 SEU CONTRATO COM A AWS

1.1 **Este Contrato.** Este conjunto de termos e condições (este "Contrato") é um contrato celebrado entre você (ou a Pessoa Jurídica para a qual você trabalha) ("você") e a Amazon Web Services, Inc. ou outra pessoa jurídica indicada na Cláusula 10 (em ambos casos, "AWS" ou "nós"). Este Contrato rege sua participação no Programa de Ensino da AWS (o "Programa") descrito na página <https://aws.amazon.com/education/awseducate/> e em suas subpáginas (o "Site do Programa"), incluindo o seu (a) uso e envio de dados, texto, áudio, vídeo, imagens, software (inclusive imagens de máquina), ou outros materiais (coletivamente, o "Conteúdo") em conexão com o Programa; e (b) uso de quaisquer das ferramentas, websites e serviços que a AWS poderá oferecer a você em conexão com o Programa (coletivamente, as "Ferramentas do Programa"). Se você estiver celebrando este Contrato em nome de uma entidade comercial, instituição governamental ou qualquer outra entidade ("Entidade"), tal como a empresa ou instituição de ensino para a qual você trabalha, você declara que tem capacidade legal para celebrar contratos em nome desta Entidade e as referências a "você" neste Contrato serão consideradas como referências à respectiva Entidade. Se você já possuir um Contrato do Cliente AWS (disponível na página <http://aws.amazon.com/agreement/>) ou outro contrato entre você e a AWS regendo seu uso dos serviços prestados pela AWS ("Contrato de Prestação de Serviços da AWS"), esse contrato regerá seu uso dos serviços online descritos nos Termos do Serviço do referido contrato e em quaisquer outras Ofertas de Serviço reguladas por (e conforme definido) no documento.

Você deve percorrer toda a página de Termos e Condições antes de aceitar ou recusar.

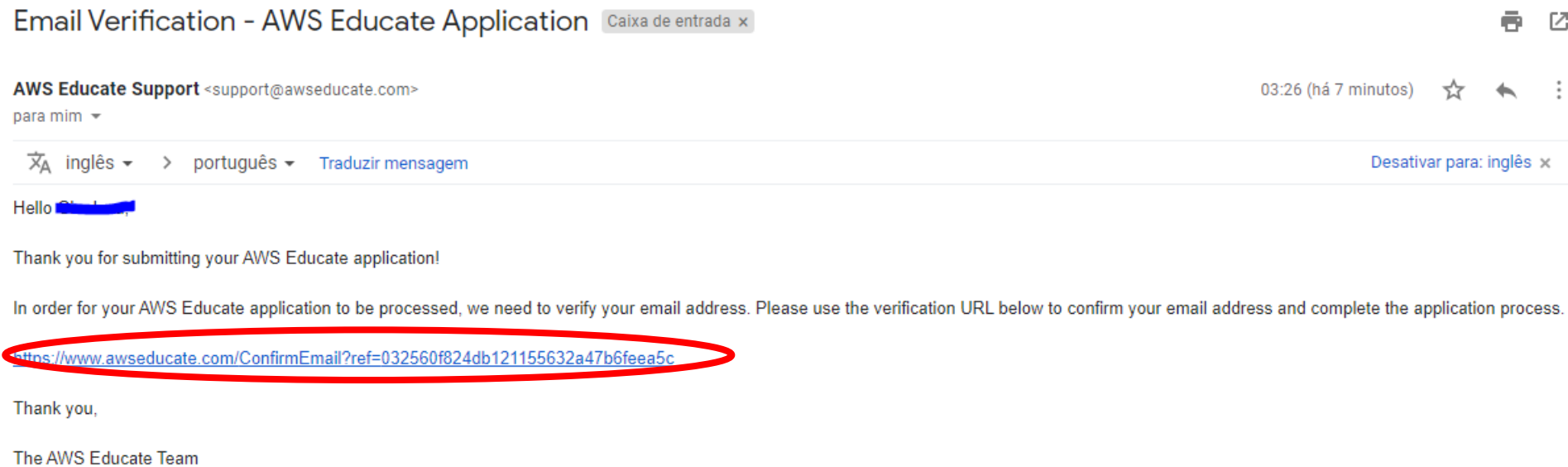
Eu
aceito

Eu
recuso

SUBMIT



- ✓ AWS – 4º Passo
 - ✓ Você receberá um email de confirmação



- ✓ AWS – 5º Passo
 - ✓ Você receberá um email se a conta for aprovada.
 - ✓ Um link para o cadastro da senha

AWS Educate Application Approved Caixa de entrada x 🖨️ 🔗

AWS Educate Support <support@awseducate.com> 03:30 (há 5 minutos) ☆ ↶ ⋮
para mim ▾

🌐 inglês ▾ > português ▾ Traduzir mensagem Desativar para: inglês x

Dear **Olivia**,
Congratulations!

Your AWS Educate application has been approved. As a member of the AWS Educate program, you will gain access to the benefits listed below:

AWS Educate Student Portal
The AWS Educate Student Portal is the hub for AWS Educate students around the world to find AWS content to help with classwork, connect to self-paced labs and training resources.

Click here to set your password and log in to the AWS Educate Student Portal. After logging in, click “AWS Account” at the top of the page to access your AWS Educate Starter Account. Use this feature to gain access to the AWS Console and resources, and start building in the cloud.

Bookmark the AWS Educate Student Portal for easy access, or [click here](#) to sign in directly.

You can access a video walk-through of the AWS Educate Student portal [here](#).

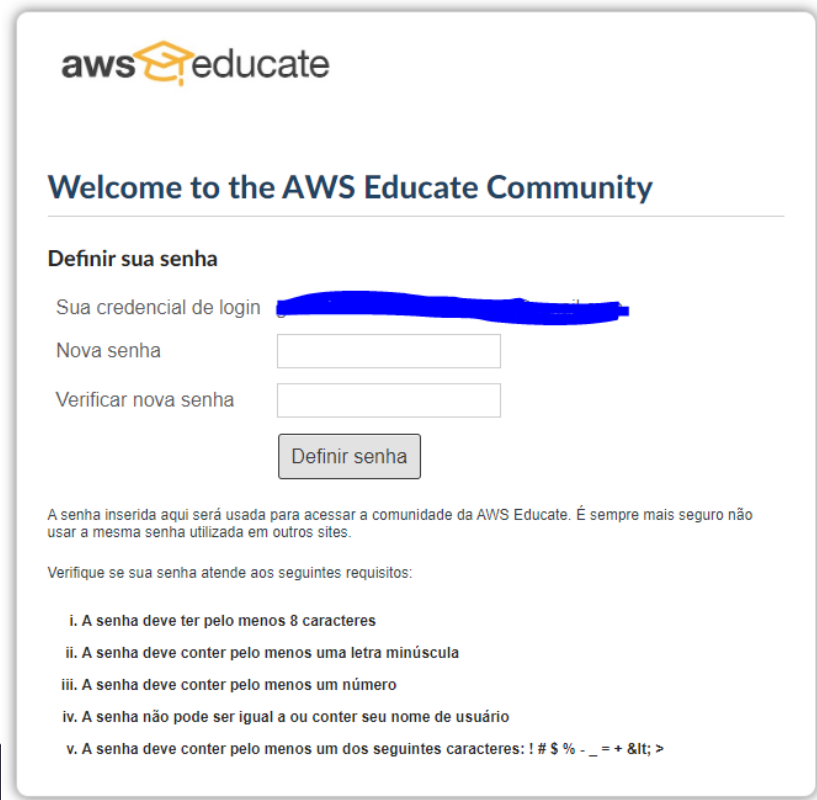
Free AWS Essentials Training
To access our foundational AWS Cloud Practitioner Essentials online learning class for free and find other self-paced labs, you must have either an AWS account or an Amazon ID.

- If you have an AWS account, sign in and [click here](#) to receive these benefits.
- If you do not have an AWS account, [click here](#) and follow the instructions to create an Amazon ID to access these benefits.

Once you access the Training and Certification portal, click “Learning Library” and search for “AWS Cloud Practitioner Essentials” to easily locate and enroll in AWS Cloud Practitioner Essentials on-line training. You can access AWS training any time after setting up your account by clicking [here](#).

Thank you again for participating in AWS Educate and we hope you enjoy the program!

- ✓ AWS – 6º Passo
 - ✓ Definir uma senha de acesso



awseducate

Welcome to the AWS Educate Community

Definir sua senha

Sua credencial de login 

Nova senha

Verificar nova senha

A senha inserida aqui será usada para acessar a comunidade da AWS Educate. É sempre mais seguro não usar a mesma senha utilizada em outros sites.

Verifique se sua senha atende aos seguintes requisitos:

- i. A senha deve ter pelo menos 8 caracteres
- ii. A senha deve conter pelo menos uma letra minúscula
- iii. A senha deve conter pelo menos um número
- iv. A senha não pode ser igual a ou conter seu nome de usuário
- v. A senha deve conter pelo menos um dos seguintes caracteres: ! # \$ % - _ = + & !t; >

✓ AWS – 7º Passo

✓ Acesse o aws Account

[My Classrooms](#) [Portfolio](#) [Career Pathways](#) [Badges](#) [Jobs ▾](#) [AWS Account](#) [Logout](#)

Tasks Completed: 0

Badges Earned: 0

Preferred Language:
English ▾

Suggested Jobs

✓ AWS – 8º Passo

✓ Crie a conta

[My Classrooms](#) [Portfolio](#) [Career Pathways](#) [Badges](#) [Jobs](#) [AWS Account](#) [Logout](#)



I'd like to use an AWS Educate Starter Account

Choose an AWS Educate Starter Account to get access to an AWS account with a preset limit on your spend on AWS services. An AWS Educate Starter Account is run and managed by a third party (Vocareum, Inc.) and the Starter Account runs in Vocareum's environment on AWS. Starter Accounts are subject to a separate agreement between you and Vocareum under separate terms and conditions.

The AWS Educate Starter Account provides access to most but not all AWS services. Students at an AWS Educate member institution will receive up to \$100 (USD) of AWS credit per year in their AWS Educate Starter Account, and students at non-member institution will receive up to \$30 (USD) of AWS credit per year.

You don't need a credit card to use a Starter Account because AWS promotional credits are already available in the account. When your usage of AWS services exceeds the balance on the account, the account is closed and any running services or other resources on the account are lost.

Create Starter Account

- ✓ AWS – 9º Passo
 - ✓ Quase lá, se a turma estiver aberta, acesse a turma



AWS Educate Starter Account

Your cloud journey has only just begun. Use your AWS Educate Starter Account to access the AWS Console and resources, and start building in the cloud!

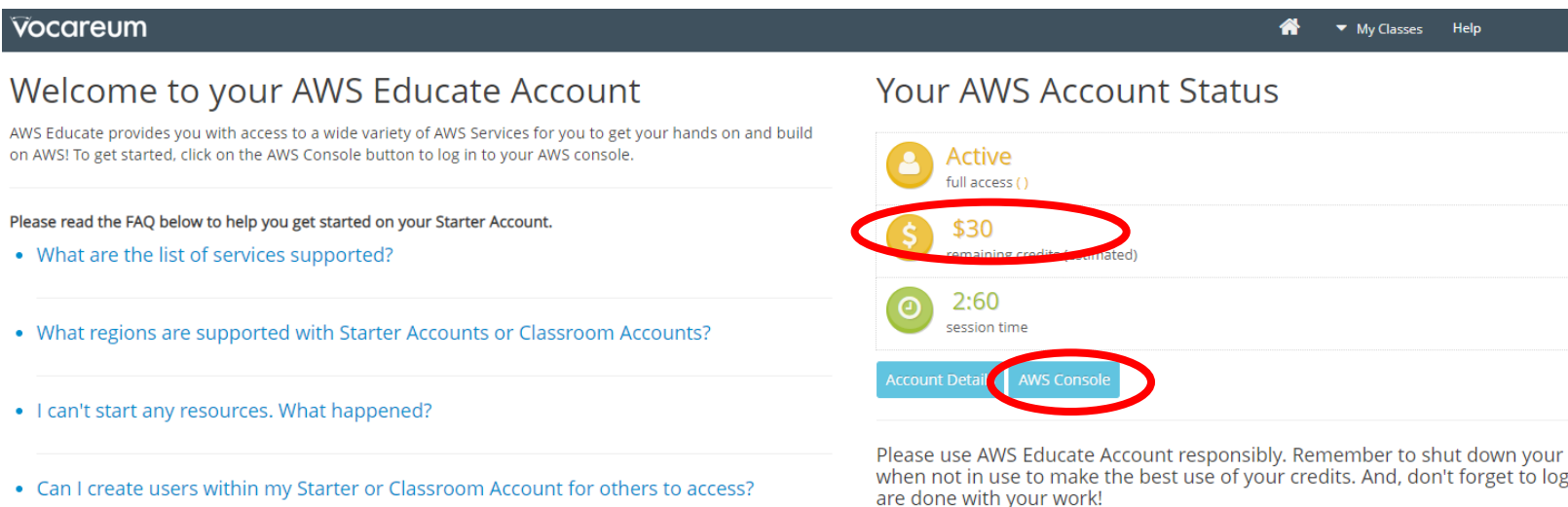
[AWS Educate Starter Account](#)

Your account has an estimated **30** credits remaining and access will end on **Jul 10, 2021**.

Note: Clicking this button will take you to a third party site managed by Vocareum, Inc. ("Third Party Servicer"). In addition to the AWS Educate terms of service, your use of the AWS Educate Starter Account is governed by the Third Party Servicer's terms, including its Privacy Policy. AWS assumes no responsibility or liability and makes no representations or warranties regarding services provided by a Third Party Servicer.

✓ AWS – 10º Passo

✓ Tudo pronto



Vocareum

Welcome to your AWS Educate Account

AWS Educate provides you with access to a wide variety of AWS Services for you to get your hands on and build on AWS! To get started, click on the AWS Console button to log in to your AWS console.

Please read the FAQ below to help you get started on your Starter Account.

- What are the list of services supported?
- What regions are supported with Starter Accounts or Classroom Accounts?
- I can't start any resources. What happened?
- Can I create users within my Starter or Classroom Account for others to access?

Your AWS Account Status

Active	full access ()
\$30	remaining credits (accumulated)
2:60	session time

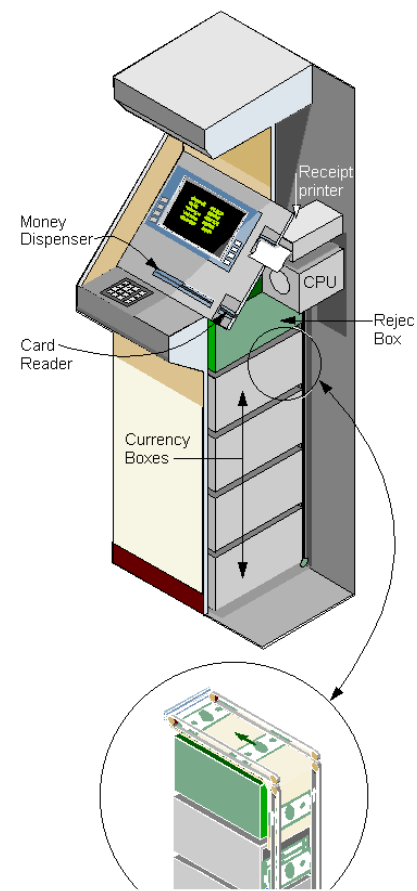
Account Details | **AWS Console**

Please use AWS Educate Account responsibly. Remember to shut down your instances when not in use to make the best use of your credits. And, don't forget to log out when you are done with your work!

Instruções

- ✓ A criação da infraestrutura será vista mais a frente
- ✓ Conhecimentos necessários Docker, ssh, etc

Porque Testamos?



Softwares podem matar!

- Em 1983 uma terceira guerra mundial quase foi deflagrada por uma falha no software de alerta de radares soviéticos. A falha fez com que o software mostrasse que os E.U.A. tinham lançado cinco mísseis.
- A guerra não aconteceu, pois o Tenente Coronel Stanislav Petrov considerou que a informação estava errada e não ordenou o contra-ataque, evitando assim o início da III guerra mundial.

Softwares causam prejuízos!

- Por uma falha em um sistema de software para controle de ações em uma bolsa de valor, a empresa americana Knight sofreu perdas de \$ 460 milhões de dólares em apenas 45 minutos.

O Que pode acontecer quando a gente não testa nosso software?

- ✓ Bancos perderem milhões
- ✓ Telefones pararem de funcionar
- ✓ Aviões terem suas rotas desviadas
- ✓ Sistemas clínicos causarem problemas a pacientes

Importância do teste de software para os negócios

- ✓ Estabelecemos um contrato com nosso cliente
- ✓ Falhas geram frustração
- ✓ Sempre o cliente encontrará o erro

O Que pode acontecer para a empresa que desenvolve o software?



O Que pode acontecer para a empresa?



Conceitos

Vamos parar de gambiarra!



Mas afinal o que é teste?

É a verificação feita sobre um sistema ou parte dele para garantir que uma determinada entrada produza, sempre, uma saída esperada.

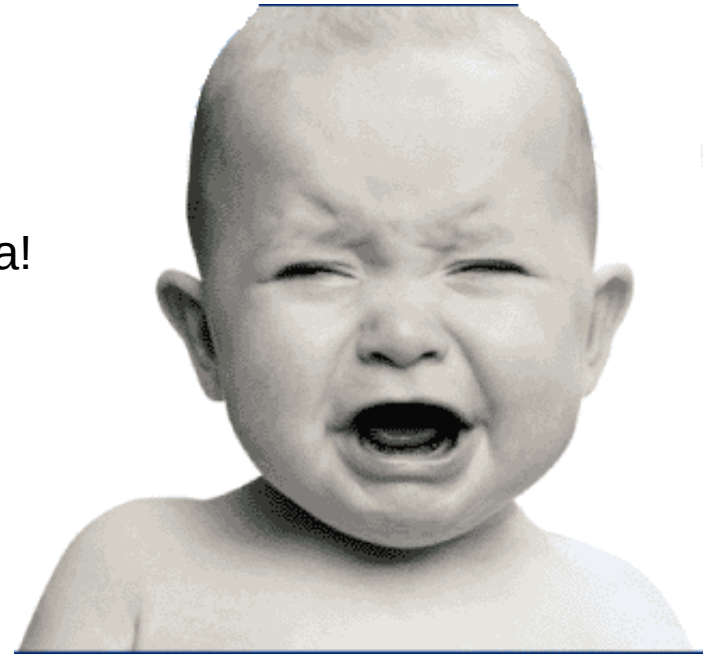
E por que as pessoas não testam?

Porque demora!

Meu projeto está atrasado!

Porque sou programador, e programador não testa!

Compilou, tá funcionando!



Tá pronto... Só falta testar!

Algumas verdades importantes

- ✓ Testes não verificam completamente as saídas de um sistema, pois as entradas são infinitas
- ✓ Testes não garantem qualidade
- ✓ Testes custam em média de 20% a 30% do custo total de elaboração de um software

O perfil dos profissionais

- ✓ O mercado tem pouquíssimos testadores especialistas. Todos querem ser programadores!
- ✓ Dos testadores, a maioria não são realmente especialistas, apenas foram inseridos em alguma equipe de teste da empresa
- ✓ Existe um pouco de preconceito com testes, que é falado como uma “parte chata do desenvolvimento de software”
- ✓ Encontrar um profissional realmente com o perfil, conhecimentos e habilidades de um testador é algo realmente raro

Definições Iniciais

- ▶ Engenharia de Software:
 - Conjunto de princípios, métodos e técnicas que tratam o software como produto de engenharia que requer planejamento, projeto, implementação e manutenção.
 - Objetivo Principal:
 - Produzir software dentro do prazo, no custo estimado e a qualidade esperada pelo cliente.

Definições Iniciais

- ▶ **Qualidade de Software:**
 - Conformidade com especificações e padrões de desenvolvimento documentados.
 - Atendimento das necessidades dos usuários.

- ▶ **Garantia de Qualidade do Software:**
 - Conjunto de **atividades técnicas** aplicadas durante **todo** o processo de desenvolvimento.
 - O objetivo é garantir que tanto o **processo de desenvolvimento** quanto o **produto de software** atinjam níveis de qualidade especificados.

Teste de software

- “Teste de software é o processo de executar o programa com a intenção de encontrar bugs”

- **Glendford Myers, 1979**

- **Um bug encontrado durante a especificação pode custar 1 real. O mesmo bug encontrado no release pode custar 100x mais**

O Que teste de software não é?!

- ✓ Testes ajudam a encontrar erros, mas não corrigí-los.
- ✓ Cabe aos desenvolvedores fazerem a correção

Dificuldades na criação de softwares

Softwares são complexos

Satisfazer o cliente é difícil...



Levam vários meses/anos para serem desenvolvidos

Várias pessoas trabalhando no mesmo produto

A comunicação é um problema...

Software muda

Prazos e recursos são limitados



Como minimizar os problemas

- ✓ Atender as **necessidades** do cliente
- ✓ **Testes contínuos** no software
- ✓ **Documentação** do sistema
- ✓ Adoção de **boas práticas** de desenvolvimento
- ✓ Construir software com **qualidade!**

Qualidade na computação

- **Conformidade** com requisitos funcionais e não funcionais, **padrões** de desenvolvimento **documentados** e características implícitas **esperadas** de todo software profissionalmente desenvolvido

Como garantimos a qualidade de nosso software?

Criando uma documentação consistente

Validação e verificação

Adotar padrões de projeto

Admitir que há erros e tentar encontrá-los



Como encontrar erros no software

- ✓ Inspecionar nosso Código, documentos, etc
- ✓ Avaliar se o software atende aos requisitos do usuário
- ✓ Pedir para o cliente verificar
- ✓ Técnicas de Testes, Caixa branca e Preta

Como correm os defeitos no processo de software

Falha humana

Problema de comunicação



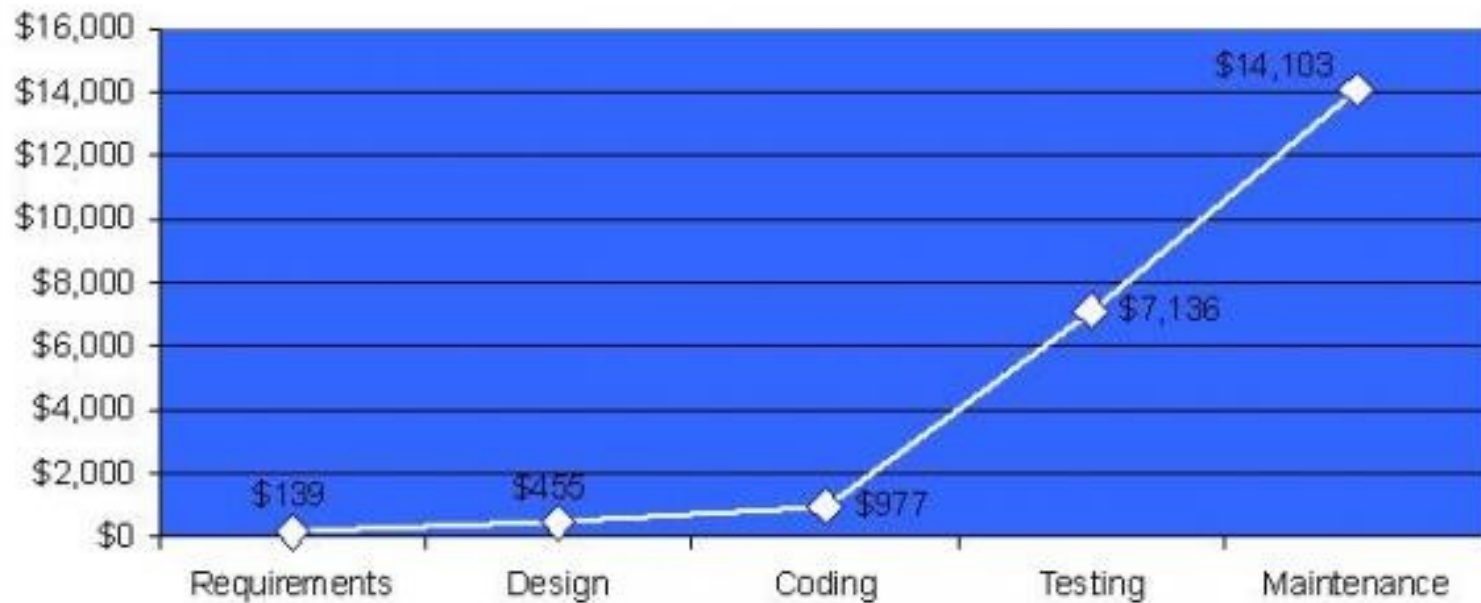
Conceitos

Por que estudamos teste?

- ✓ Mais de 1/3 das falhas poderiam ser evitadas com testes
- ✓ Cerca de 50% das falhas só são descobertas em produção
- ✓ Segundo uma pesquisa do Departamento de Comércio dos EUA, publicada em 2012, falhas de software são tão comuns e tão danosas que se estima que causem um prejuízo anual de mais de 60 bilhões de dólares para a economia americana.

Por que estudamos testes?

Cost Per Defect



O Que percebemos?

- ✓ Falhas de software causam grandes prejuízos
- ✓ A prática de testes ainda é pouco utilizada
- ✓ Quanto antes a presença do defeito for revelada, menor o custo da correção e maior a probabilidade de corrigí-lo
- ✓ Testes não encontram todas as falhas, mas ajudam a minimizar os erros

Quando utilizar testes?

- ✓ Desejamos oferecer uma melhor qualidade ao código
- ✓ Garantir que os requisitos estão implementados e estão corretos
- ✓ Fornecem segurança para realizar alterações no código

O Que são testes de software?

- ✓ São previsíveis
- ✓ São finitos
- ✓ São (ou deveriam ser) simples
- ✓ Não são perfeitos, mas uma das melhores alternativas para validação

Pensamentos Importantes!!!

Inspeções aumentam significativamente a produtividade, qualidade e estabilidade dos projetos (Fagan, 1976).

Inspeções e Testes encontram diferentes tipos de defeitos (Hetzl, 1976 & Meyer, 1978).

Qualidade melhora a produtividade (Travassos e Kalinowski, 2008).

Prevenção de defeitos é melhor do que remoção de defeitos (Mays, 1990).

Testes podem mostrar a presença, mas não a ausência de defeitos (Dijkstra, 1970).

Pensamentos Importantes!!!

Um desenvolvedor não é indicado para testar seu próprio código (Weinberg, 1971).

Aproximadamente 80% dos defeitos são provenientes de 20% dos módulos (Pareto, 1897) (Endres, 1975).

Usabilidade é quantificável (Nielsen, 1994).

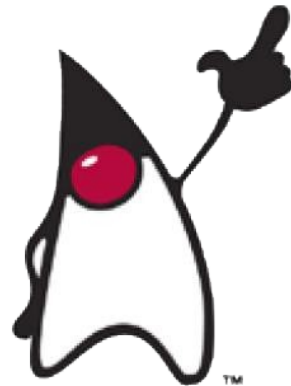
Corrigir um defeito após a entrega do produto é freqüentemente 100 vezes mais caro do que corrigi-lo durante as atividades de requisitos e projeto do sistema (Boehm, Basili, 2001).

Teste de Software

Introdução



Testes no desenvolvimento



Professor Daves Martins

Doutorando em Eng. Elétrica

Mestre em Computação de Alto Desempenho pela UFRJ

Especialista em Banco de Dados

Analista Web – UFJF

Professor Efetivo – IFET JF

Sumário da Aula

Tipos de
Teste

Pirâmide
de Teste

Boas
Práticas

Inspeção

Dúvida de Aluno

Importante

- ✓ **Validação:** assegurar que o produto final corresponda aos requisitos do usuário
 - ✓ Estamos construindo o produto certo?
- ✓ **Verificação:** Assegurar consistência, completitude e corretitude do produto em cada fase e entre fases consecutivas do ciclo de vida do software
 - ✓ Estamos construindo corretamente o produto?
- ✓ **Teste:** Examina o comportamento do produto por meio de sua execução

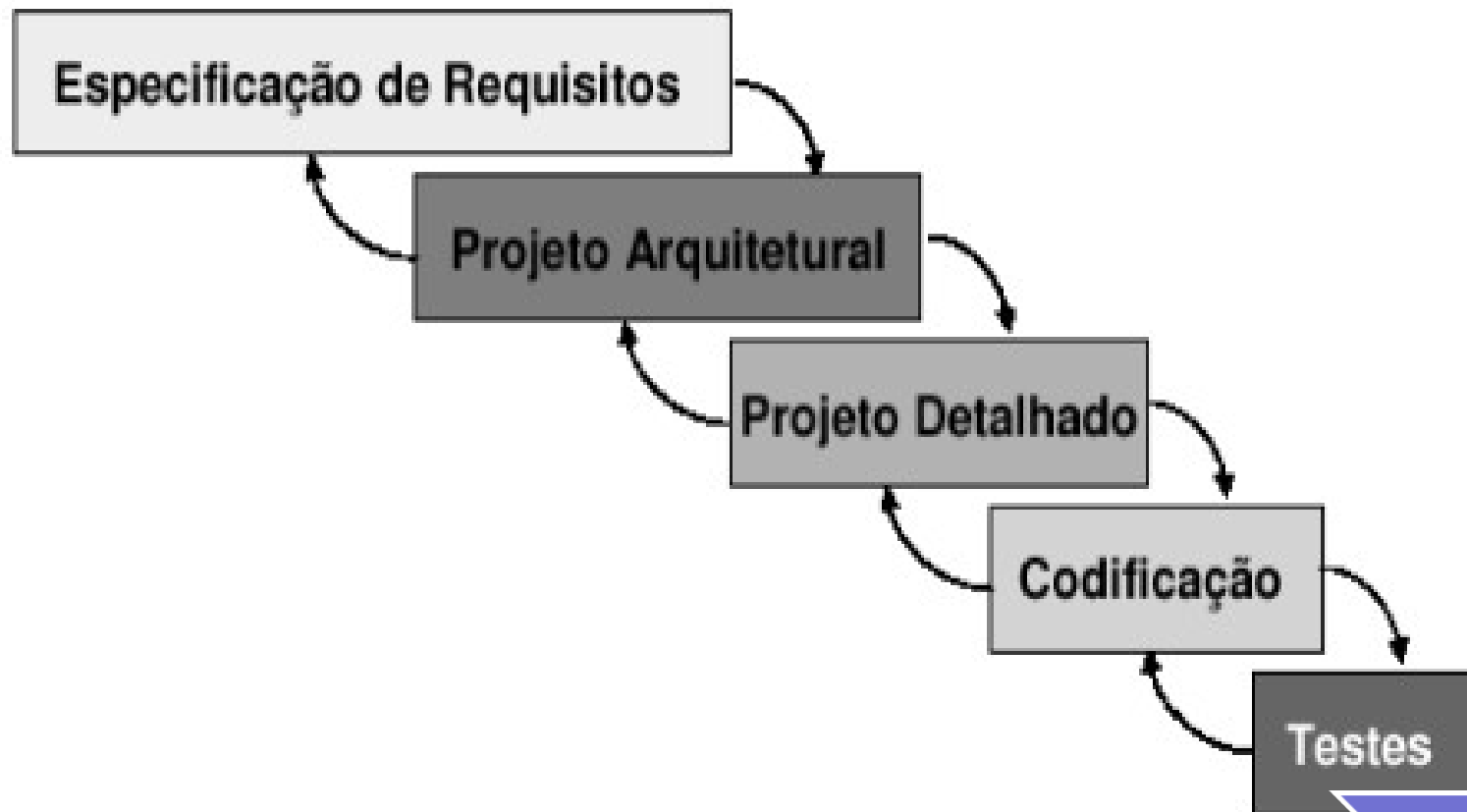
Porque Utilizar Métodos de VV&T?

- ▶ A utilização destes métodos na indústria têm mostrado resultados positivos considerando tanto **produtividade** quanto **qualidade**.
- ▶ Resultados de estudos experimentais evidenciam **benefícios** da utilização destes métodos no desenvolvimento de software.
- ▶ Diversas **evidências** relacionadas a VV&T foram obtidas ao longo dos anos de pesquisa em engenharia de software.

O Que é teste de software?

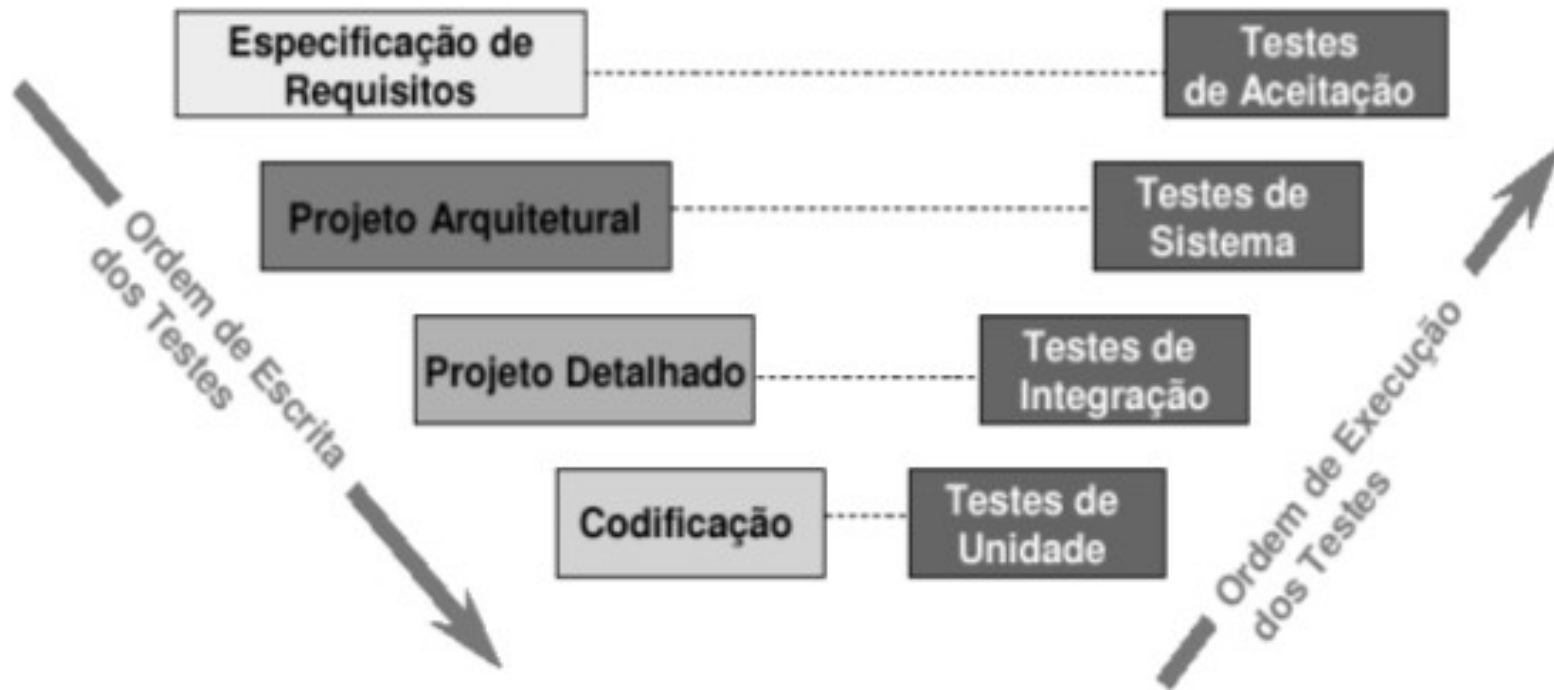
- ✓ Analisar documentos e verificar se foi produzido corretamente
- ✓ Testar funções e verificar se as entradas produzidas resultam nas saídas esperadas
- ✓ Fazer uso do software desenvolvido e verificar se está conforme os requisitos

■ Ontem



Tipos de
Teste

■ Hoje



Níveis de Teste

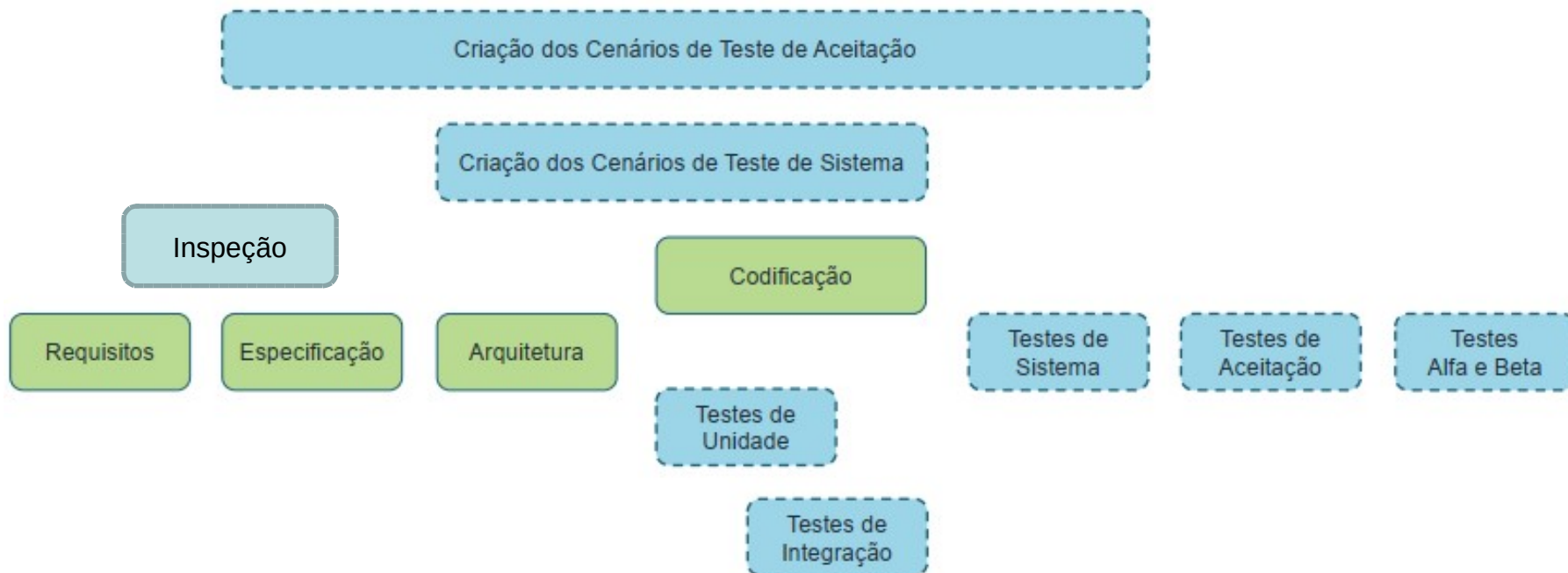
Quando Testar?

Como Testar?

O que Testar?

Tipos de
Teste

Quando e Como os Testes Entram no Processo de Desenvolvimento de Software



Tipos de Teste

Quando Testar?

Níveis de Teste

Analise

- Inspeção Ad-hoc
- Inspeção checklist

Projeto

- Planejamento
- Plano de Teste
- Caso de Teste

Código

- Teste de Regressão
- Teste de Cobertura
- Teste de Sistema ou Funcional
- Teste de Integração
- Teste de unidade

Testes

- Teste Beta
- Teste Alfa
- Teste de Aceitação
- Teste de Sistema

Deploy

- Teste de Segurança
- Teste de Stress
- Teste de Desempenho

Tipos de
Teste

Tipos de Teste

Como Testar?

Técnicas de Teste

Caixa Preta (Funcional)

- Não precisa de acesso ao código
- Baseado em entradas previamente definidas e saídas esperadas.
- Equipe de Teste
- Particionamento de equivalência, Análise de valor limite, Tabela de decisão

Caixa Branca (Estrutural)

- Acesso ao código fonte
- Feito por programador
- Teste de unidade, etc
- Análise estática do código (Jacoco, PMD, findBugs)

Tipos de
Teste

Tipos de Teste

Transparência do Teste

Teste de caixa branca

Teste baseado na análise da estrutura interna de um componente ou sistema.

(Glossário ISTQB)

Código Fonte e Arquitetura (Estrutura)

Requisitos / Negócios (Abstração)

Teste de caixa preta

Teste, funcional ou não funcional, sem referência à estrutura interna do componente ou do sistema.

(Glossário ISTQB)

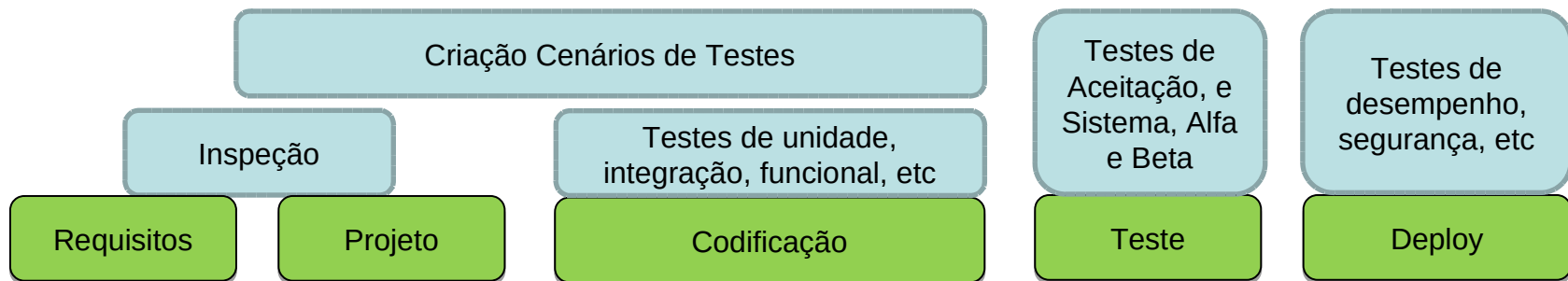
Tipos de Teste



Teste de Software

“O software sempre será testado. Pode ser testado por você ou será testado pelo seu cliente/usuário.”

Quando e Como os Testes Entram no Processo de Desenvolvimento de Software



Teste de software

Problemas ocorridos na utilização de produtos de software, percebidos muitas vezes pelo usuário final, podem ser decorrentes de falta de qualidade no processo de desenvolvimento, onde técnicas de inspeção e de testes são fortes aliadas na identificação precoce dos defeitos, melhorando a satisfação do usuário e minimizando os elevados custos associados a manutenções corretivas

Uma das formas de melhorar a qualidade na construção de software é a implementação de procedimentos de teste em todas as fases de seu ciclo de vida

Entretanto, esta é uma prática pouco utilizada nas empresas, pois normalmente necessita de grande mão-de-obra e um esforço desregrado com resultados pouco favoráveis em muitos casos

Isso acontece, pois testes devem ser embasados em fundamentos lógicos, e não simplesmente voltados para execução de funcionalidades aleatoriamente, em tentativas tendenciosas de encontrar defeitos

Não é possível aplicá-los exaustivamente
Para aumentar sua eficácia, deve ser executado por equipes independentes

Um bom teste:

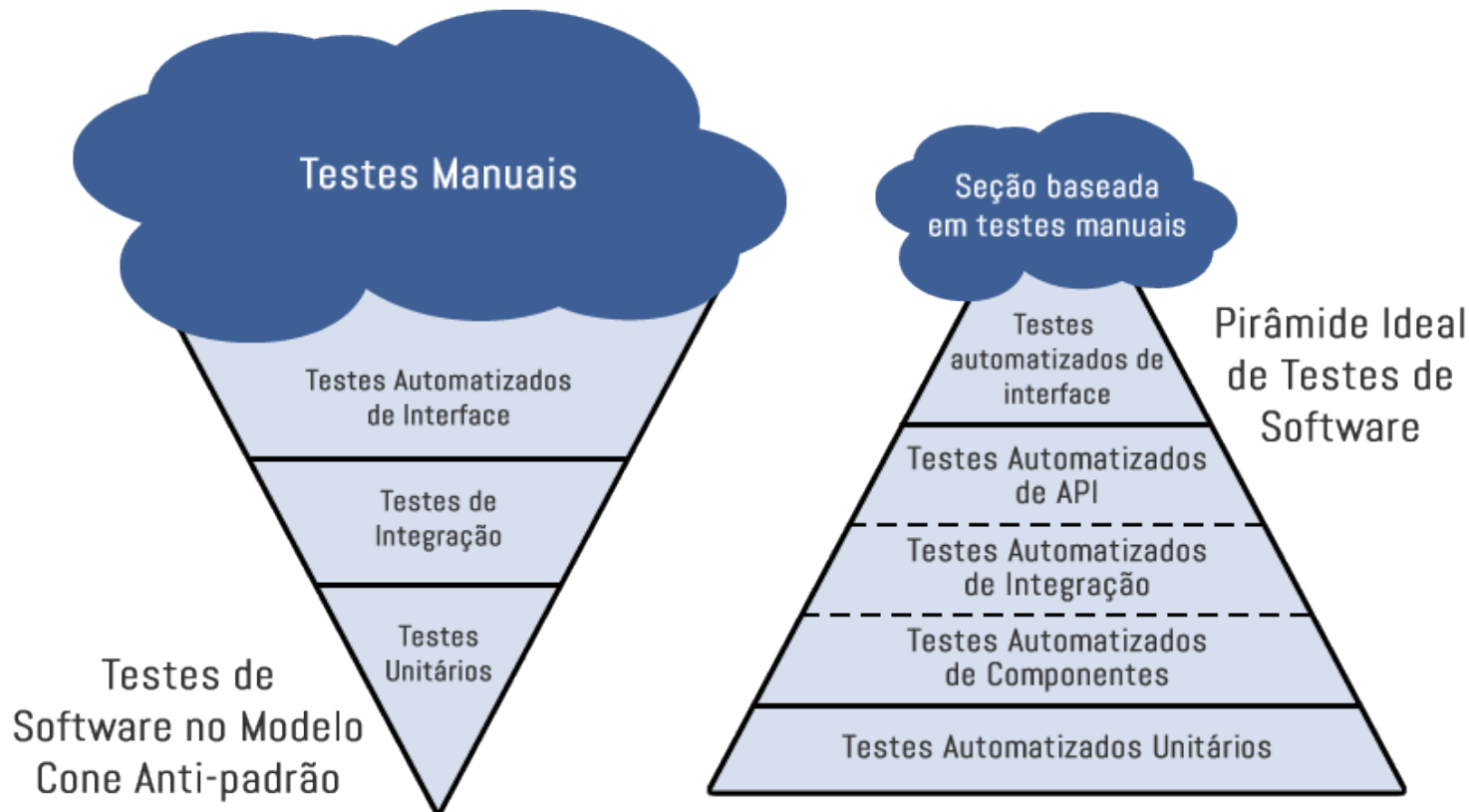
Possui alta probabilidade de encontrar um defeito

Não é redundante

É abrangente o suficiente

Possui um nível adequado de complexidade

Pirâmide de Teste



Testes automatizados de interface devem representar o menor volume da automação. O maior benefício da automação de testes é gerado com um maior volume de testes unitários.

Pirâmide
de Teste

- ✓ A pessoa que especifica os cenários de teste de sistema não deve estar participando da codificação do sistema
 - De preferência, que seja um testador especialista e não um programador
 - A especificação é a base para os cenários de teste
 - “O que não está especificado, não será validado”
- ✓ Faça uma boa seleção de cenários, considerando:
 - O tempo disponível para execução dos testes
 - A maturidade do(s) testador(es)
 - A relevância das funcionalidades a serem testadas
 - Lembre-se: os cenários são infinitos. Nunca haverá tempo para testar todos.

- ✓ Seja pessimista! Cuidado para não ignorar cenários aparentemente inocentes
- ✓ Registre todas as falhas encontradas e acompanhe até que seja corrigida

- ✓ Açou uma falha? Detalhe ao máximo!
 - ✓ “A caixa de mensagens não segue o padrão”
 - ✓ “Após não preencher um dos campos obrigatórios no formulário de cadastro de usuário, a caixa de mensagens apresentada está em um formato diferente das demais caixas de mensagens de erro do sistema”
 - ✓ Browser: Chrome v10.2
 - ✓ Dispositivo: Tablet Ipad 7 (colocar printscreen)
- ✓ A falha foi corrigida? Reteste! (teste de regressão)

- ✓ No lugar planilhas, arquivos textos, você pode optar por ferramentas web para gestão de incidentes/falhas/melhorias identificas

- ✓ Teste sempre os valores limites, grande parte dos erros estarão lá!
- ✓ Formulários:
 - ✓ Campo em branco
 - ✓ Campo com valor inválido
 - ✓ Quantidade de caracteres
 - ✓ E-mail inválido
 - ✓ Mensagens de erro coerentes
 - ✓ Cálculos:
 - ✓ Valores positivos
 - ✓ Valores negativos
 - ✓ ZERO
 - ✓ Intervalos fechados e abertos
 - ✓ Acima do limite máximo e abaixo do limite mínimo
 - ✓ O mais importante: tente pensar como um usuário pensaria

Sumário da Aula

Inspeção

Tipos de
Inspeção

Boas
Práticas

Exemplo
de
Inspeção

Dúvida de Aluno

Importante

Tem sido o tipo de revisão de software mais estudado e utilizado

Objetivos

- Identificar erros específicos em documentos e em sistemas
- Identificar erros sistemáticos no processo de desenvolvimento
- Identificar desvios em relação às especificações e padrões

Benefícios

Provê ganhos significativos em relação a prazos e custos. Tende a achar mais defeitos que qualquer outro processo, e a um custo menor

No início, era aplicado apenas a código fonte.

Atualmente são amplamente utilizadas

É um método eficaz para o controle da qualidade e aumento da produtividade

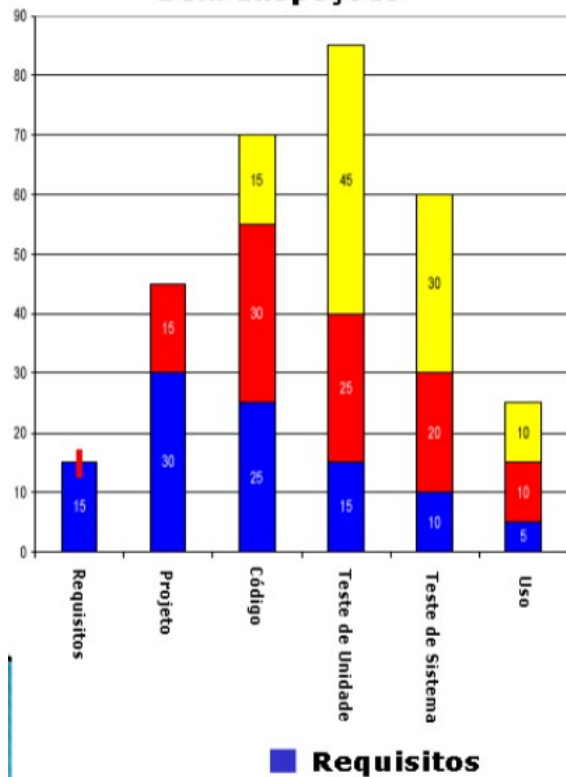
Benefícios e Custo de Inspeções:

- Inspeções vêm sendo utilizadas há mais de três décadas;
- Existe evidência experimental de sua usabilidade e adequabilidade;
- Provêem um bom meio para o gerente do projeto monitorar a qualidade e progresso do projeto;
- Apresentam baixo custo devido ao fato do revisor não precisar investir muito tempo ou mesmo não demandar ferramentas sofisticadas para realizá-las.

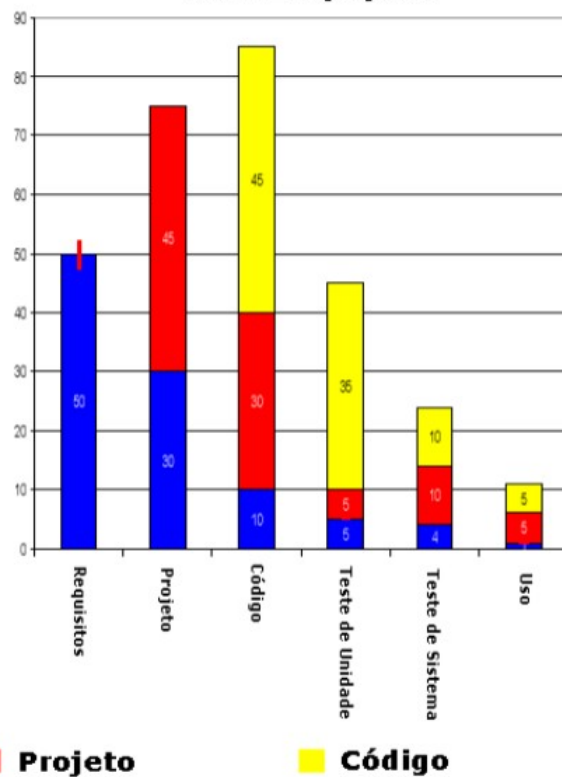
Uma alta taxa de atividades de inspeção ao longo do processo pode representar de 5% a 10% do esforço de desenvolvimento.

- Inspeções em requisitos e projeto, conduzidas no JPL
 – Nasa Jet Propulsion Laboratory (Miller,1990);
- Identificação mais cedo de defeitos.

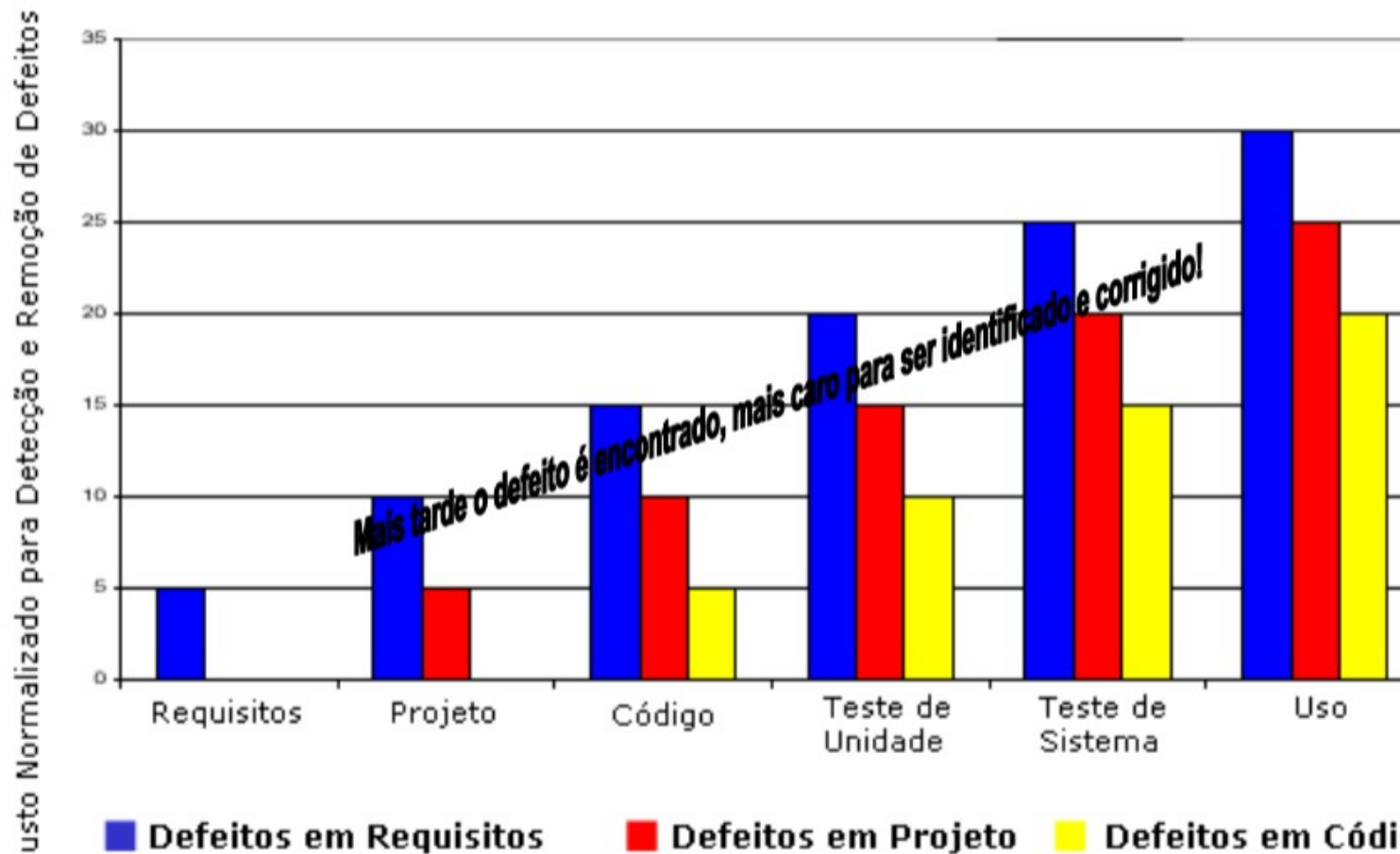
Sem Inspeções



Com Inspeções

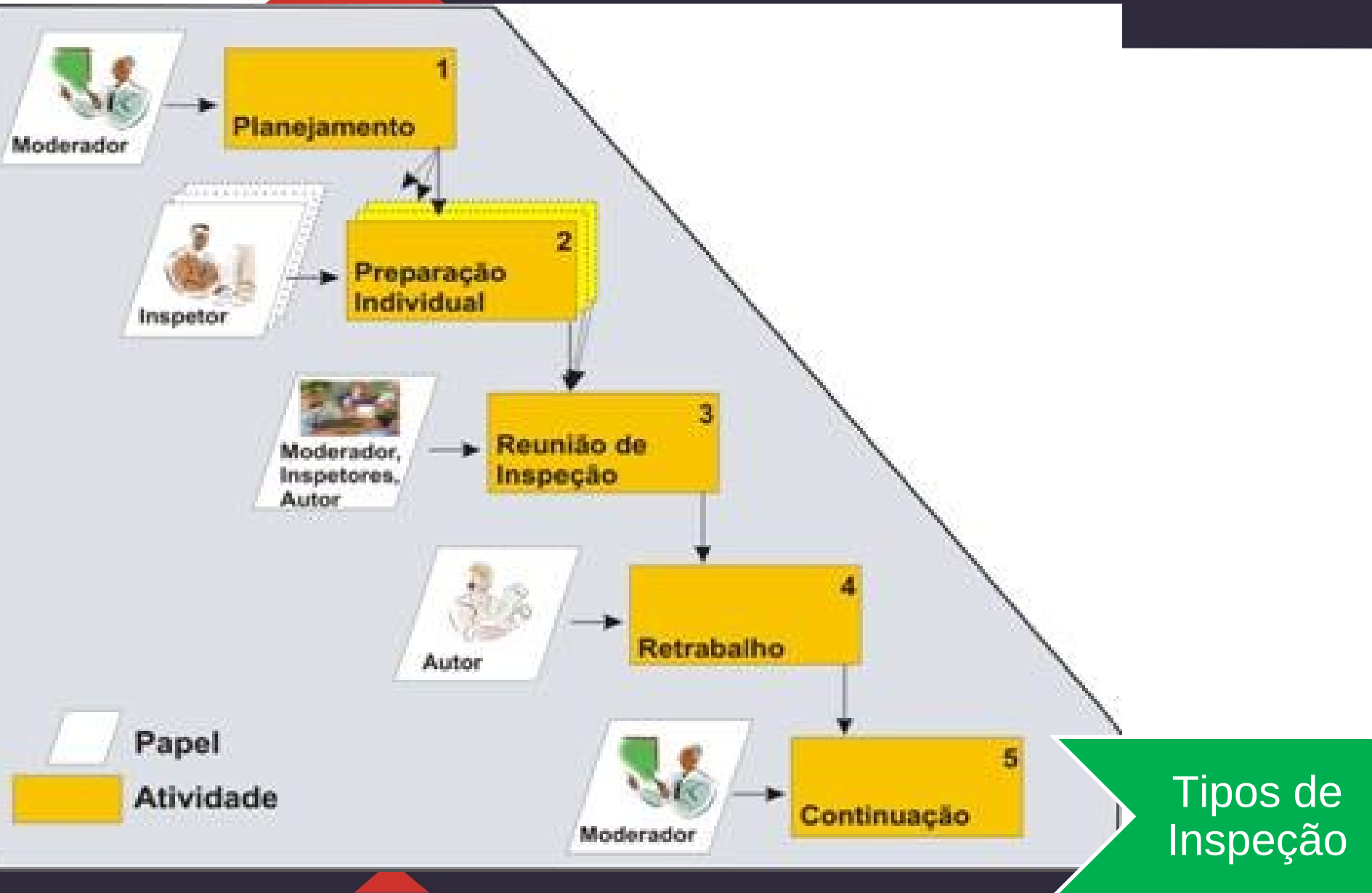


Inspeção de Requisitos



Inspeção

Processo de inspeção de software



Inspeção Ad-Hoc

- Inspetor lê o documento de acordo com sua perspectiva
- Experiência individual afeta o resultado final:
 - Foco reside na especialidade do inspetor
 - Produtividade individual
 - Difícil garantir que inspetor leu o documento de forma adequada, pois cada um aplica "sua própria técnica"
- Não existe garantia de cobertura do documento como um todo
- Custo/Eficiência (# defeitos/tempo) pode ser adequado quando inspetores possuem experiência alta (> custo)

Inspeção com Checklists

- Inspetor segue uma lista de itens com características a serem revisadas, mas ainda aplica leitura *Ad-hoc* para identificar os defeitos
- Resultado final mais direcionado:
 - Características de qualidade definidas à priori
 - Produtividade individual
 - Difícil garantir que inspetor leu o documento de forma adequada, mesmo tendo sido definidas as características de qualidade a se procurar
- Cobertura do documento relacionada aos itens do *Checklist*
- Custo/Eficiência depende do *Checklist* e dos inspetores
- *Checklist* pode ser adaptado ou construído para capturar uma determinada especificidade

Lista de Verificação de Inspeção de Requisitos

Os requisitos exibem uma clara distinção entre funções e dados?

Os requisitos definem todas as informações a serem exibidas aos usuários?

Os requisitos definem as respostas ao sistema e ao usuário às condições de erro?

É cada requisito indicado claramente, conciso, e não ambíguo?

É cada requisito testável?

Há requisitos ambíguos ou inconsistentes?

Há conflitos de requisitos?

Existem áreas não identificadas na especificação de requisitos de software que necessitam ser?

Os requisitos de desempenho (tais como o tempo de resposta, requisitos de armazenamento de dados) são indicados?

Se os requisitos envolverem complexas cadeias de decisão, estão expressos em um formato que facilite sua compreensão (isto é, tabelas de decisão, árvores da decisão, etc..)?

Os requisitos para executar melhorias do software foram especificados?

Há requisitos que contém um nível desnecessário de detalhe do projeto?

As restrições de tempo real foram especificadas em detalhe suficiente?

A precisão e a exatidão dos cálculos foram especificadas?

É possível desenvolver um conjunto completo de testes baseados nas informações contidas nas especificações de requisito do software? Se não, que informação falta?

As suposições e as dependências foram indicadas claramente?

O documento contém toda a informação identificada no escopo para a especificação de requisitos do software?

Tipos de Defeitos de Inspeção

Defeito	Descrição Geral
<i>Omissão</i>	Informação necessária sobre o sistema foi omitida do artefato de software
<i>Fato Incorreto</i>	Alguma informação no artefato de software contradiz informação do documento de requisitos ou o conhecimento geral do domínio.
<i>Inconsistência</i>	Informação contida em uma parte do artefato de software está inconsistente com outra informação no artefato de software
<i>Ambiguidade</i>	Informação contida no artefato de software é ambigua, isto é, várias interpretações podem ser derivadas da definição levando o desenvolvedor a implementação correta
<i>Informação Estranha</i>	Informação que é fornecida não é necessária ou nunca utilizada

Relato de Defeitos em Inspeção de Requisitos

Formulário para Relato de Defeito

Nome: J.J. XPT

Checklist utilizado: 01

Documento Revisado: Doc. De Requisitos da Ferramenta de construção de Casos de Uso para utilização na Técnica de Leitura PBR.

Tempo utilizado: 2 hs

Defeito No.	Página No.	Req. No.	Tipo Defeito	Descrição
1	2	RF 8	Omissão	Falta uma forma de consulta aos elementos do modelo presentes na ferramenta. Exemplo: estrutura em árvore ou diretórios
2			Omissão	Os requisitos não abordam tratamento de erros
3	3	RF 11/12	Ambigüidade	Não está muito clara a diferença entre os requisitos 11 e 12
4	2	RF 5	Ambigüidade	Usa-se no documento os termos participante e ator para descrever a mesma coisa.
5			Omissão	Falta descrição de como será a interface com o usuário e sua navegação.

Tipos de
Inspeção

Defeitos em Requisitos

- Exemplo: Sistema de Controle para Postos de Gasolina

Requisito Funcional 5

- Se o pagamento for feito em dinheiro, o caixa é responsável por receber o pagamento e, caso necessário, devolver o troco. Quando o pagamento for feito o caixa fornece essa informação ao sistema. O sistema e a bomba de combustível retornam então para seu estado inicial.

Qual o valor da compra?

- Para poder completar a transação, o caixa precisa saber o valor da compra.
- Informação foi perdida durante a criação dos requisitos.
- Esta informação foi omitida da descrição da funcionalidade, logo temos um defeito.

Defeitos em Requisitos

- Requisito Funcional 3
 - Se o cliente optar pelo pagamento à vista, ele poderá pagar em dinheiro ou com cartão de crédito. Se escolher pagar em dinheiro, a bomba de combustível deve enviar mensagem instruindo o cliente a procurar o caixa. Se escolher pagar com cartão de crédito, a bomba deve instruir o cliente a inserir o cartão no leitor. Se o cliente não fizer uma opção ou fizer uma opção inválida, o sistema assume o pagamento com cartão de crédito como *default*.

A informação não foi traduzida corretamente.

O conhecimento do domínio indica que o pagamento com cartão de crédito não deve ser a opção *default*, logo temos um defeito.

Defeitos em Requisitos

- Requisito Funcional 2
 - Após o abastecimento, a bomba de combustível informa o valor ao sistema. O valor máximo é de R\$ 999,99.
- Requisito Funcional 4
 - Se o pagamento for feito com cartão de crédito, o leitor de cartão envia o número do cartão para o sistema. Se o sistema recebe um número de cartão inválido, então uma mensagem é enviada a bomba para o cliente introduzir o cartão novamente no leitor. Ao receber um número de cartão válido, o número do cartão e o valor da compra são enviados para o sistema da operadora do cartão, e o sistema e a bomba de combustível retornam para o seu estado inicial. O valor máximo de uma compra é de R\$1.000,00

A informação foi descrita de forma inconsistente

Como o conhecimento de domínio não nos permite saber qual a informação está correta, temos um defeito.

Defeitos em Requisitos

- Requisito Funcional 6.1
 - O cliente deve informar o número de sua conta ao caixa, que cadastra a informação no sistema. Se o número de conta for válido, então o número da conta, o valor da compra e uma breve descrição do tipo de transação são registrados (*log*). Se um número inválido for registrado, uma mensagem de erro é mostrada ao caixa, que deve informar novamente o número. O caixa deve ter também a opção de cancelar a operação. Neste caso, o valor da compra é mostrado novamente juntamente com as opções de pagamento (à vista ou a prazo).

A informação pode ser traduzida incorretamente.

Que informação é armazenada?

O que significa 'tipo de transação'? Combustível/Manutenção?

Cartão/Dinheiro?

Defeitos em Requisitos

- Requisito Funcional 7

- Para pagar uma conta mensal, o cliente deve enviar o pagamento junto com o número de sua conta mensal. O caixa seleciona este tipo de pagamento e o sistema envia uma mensagem ao caixa para que o número da conta mensal, o tipo de pagamento e o valor do pagamento sejam informados. Se alguma dessas informações forem inválidas, o sistema retorna para a tela anterior. Se o tipo de pagamento for cartão de crédito, o número do cartão também deve ser informado, e então o recibo do cartão deve ser foto-copiado e armazenado com os outros recibos.

Uma informação estranha foi introduzida.

Uma informação que, apesar de poder estar correta, pode causar problemas já que não faz parte do sistema.

Como isto pode ser testado?

Exemplo

Considere o seguinte:

Um programa para somar 2 números inteiros formado com 2 dígitos.

Matematicamente existem 39601 testes possíveis de serem feitos usando este mesmo programa. Temos números entre -99 e 99, o que incluindo 0 dariam 199 números. Para cobrirmos todo o universo teríamos $199^2 = 39.601$.

Nós testamos todas as possibilidades?

Precisamos testar todas as possibilidades?

Exemplo
de
Inspeção

Exemplo

CT	Resultado esperado	Notas
99+99	198	Maior número possível
-99+-99	-198	Não houve menção a nos negativos
99+-14	85	Maior número em primeiro
-38+99	61	Maior número em segundo
9+9	18	Dois maiores dígitos
0+0	0	Programas muitas vezes falham com 0
0+23	23	Problemas com 0
-78+0	-78	Problemas com 0

Valores limítrofes é uma conhecida técnica de teste que usamos quando colocamos os valores máximos e mínimos 99,-99 nas operações de soma

Exemplo
de
Inspeção

Exemplo

Também devemos testar o comportamento do sistema nos valores não suportados ou esperados por ele.

Exemplo

CT	Resultado esperado	Notas
100 + 100	Mensagem de erro	Embora o requisito limite em dois dígitos pode ser que o programa aceite
<Enter> + <Enter>	Erro - msg	Será que aceita campos em branco
123456+0	Erro - msg	Explorar os limites do campo
1,2 + 3	3,2 ou erro de definição?	O requisito não fala em número decimal. Talvez seja um incidente
A = B		
Teclas de controle + teclas de controle	Mensagem de erro	O certo seria só aceitar valores numéricos

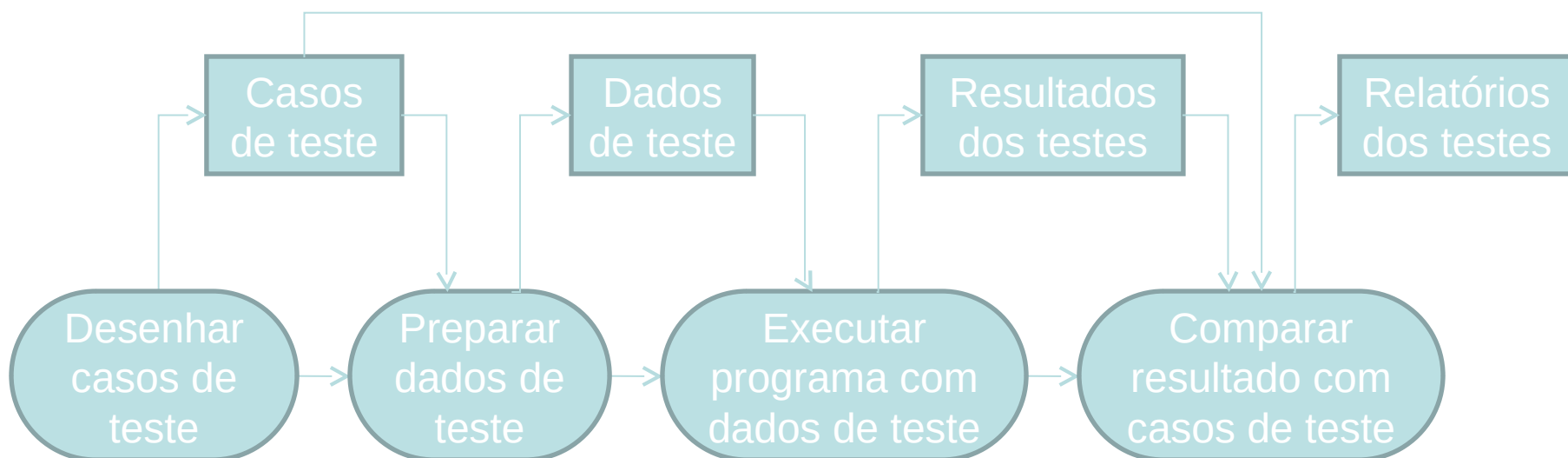
Exemplo
de
Inspeção

A INSPEÇÃO DE CÓDIGO

Este teste se caracteriza pelo fato de ser executado apenas por quem entende o código-fonte, quem vai testar precisa ser desenvolvedor também. Aqui cabem várias abordagens. Uma delas é quem fez o código é quem faz a inspeção, mas ao mesmo tempo que quem escreveu o código é a pessoa que mais o compreende, seus olhos também correm o risco de estar viciados naquilo, então uma outra abordagem é um desenvolvedor que não fez o código que o inspecione. Uma terceira é revisar em grupo. Acho esta última a mais segura.

Note que a inspeção de código é uma revisão, um highlevel checkout, você lê e tenta achar erros óbvios de lógica ou gramática/digitação. Não mais que isso. Existem outros métodos mais complexos e formais de verificação. A inspeção é importante porque é rápida em detectar erros grosseiros (ou nem tanto).

Processo de testes de *software*



Plano de Teste

ID	CT-001
Caso de Teste:	Efetuar login no sistema.
Funcionalidade:	Login
Pré - Condição:	1. Possuir usuário válido para efetuar o login.
Procedimento:	1. Acessar o sistema; 2. Inserir usuário e senha; 3. Clicar em "Login".
Resultado Esperado:	1. Login do usuário efetuado com sucesso.

ID	CT-002
Caso de Teste:	Cadastrar cliente no Sistema
Funcionalidade:	Cadastro de cliente
Pré - Condição:	Estar logado no sistema
Procedimento:	1. Entrar na tela "Cadastro" -> "Cliente" 2. Inserir dados do cliente. 3. Clicar em "Salvar".
Resultado Esperado:	1. Cliente é cadastrado com sucesso..

Planejamento na fase de testes é muito importante para garantir o sucesso e a qualidade do sistema, e para isso existem documentos que são essenciais nesta fase, são eles: Plano de teste, Caso de teste e Roteiro de teste.

Plano de Teste

Caso de teste: Descreve uma condição particular a ser testada, é composto por valores de entrada, restrições para a sua execução e um resultado ou comportamento esperado. O processo de criação de casos de testes pode também ajudar a encontrar falhas de requisitos, ou seja, identificar cenários não tratados pelos requisitos do sistema. Exemplo de casos de testes:

Normalmente associa-se inspeção a código fonte, visando melhorar sua qualidade

Utilizada para verificar:

Padrões de codificação

Estrutura dos algoritmos

Para padrões de codificação, diversas ferramentas podem apoiar essa atividade:

Checkstyle, PMD, FindBugs, Dependency Finder, SQE

Detecção de violação de código *Local Variable Names*

```
33 public String toString() {  
34     String Resultado = "(" + x + "," + y + ")";  
35     return  
36 }  
37
```

Local Variable Names: Nome 'Resultado' deve condizer com o padrão '^[a-z][a-zA-Z0-9]*\$'.
Press 'F2' for focus

Detecção de violação de código *Multiple Variable Declarations*

```
11 private double x, y;  
12  
13 public  
14 if ((x == outroPonto2D.x) && (this.y == outroPo
```

Multiple Variable Declaration: Each variable declaration must be in its own statement.
Press 'F2' for focus

Detecção de violação de código *Maximum Line Length*

```
48 pt3 = pt1.clona();  
49 System.out.println("P3 está na mesma posição de P1?" + pt3.éIgual(pt1));  
50 pt4 = pt1.origem();  
51 System.out.println("P4 está na mesma posição de P1?" + pt4.éIgual(pt1));  
52 }  
53
```

Maximum Line Length: Linha contém mais do que 80 caracteres.

Tem sido o tipo de revisão de software mais estudado e utilizado

Objetivos

- Identificar erros específicos em documentos e em sistemas
- Identificar erros sistemáticos no processo de desenvolvimento
- Identificar desvios em relação às especificações e padrões

Benefícios

Provê ganhos significativos em relação a prazos e custos.
Tende a achar mais defeitos que qualquer outro processo, e a um custo menor

No início, era aplicado apenas a código fonte.

Atualmente são amplamente utilizadas

É um método eficaz para o controle da qualidade e aumento da produtividade

Inspeção de Requisitos

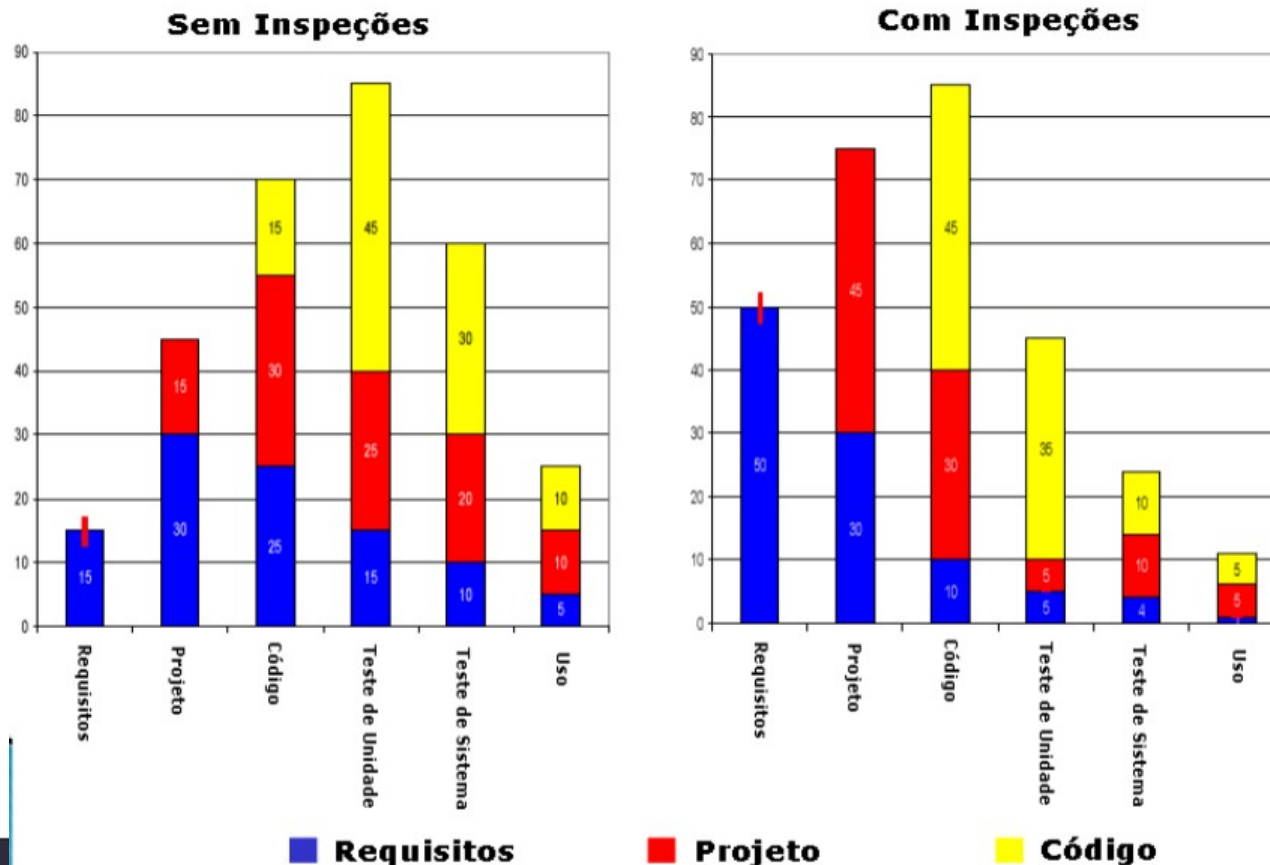
Benefícios e Custo de Inspeções:

- Inspeções vêm sendo utilizadas há mais de três décadas;
- Existe evidência experimental de sua usabilidade e adequabilidade;
- Provêem um bom meio para o gerente do projeto monitorar a qualidade e progresso do projeto;
- Apresentam baixo custo devido ao fato do revisor não precisar investir muito tempo ou mesmo não demandar ferramentas sofisticadas para realizá-las.

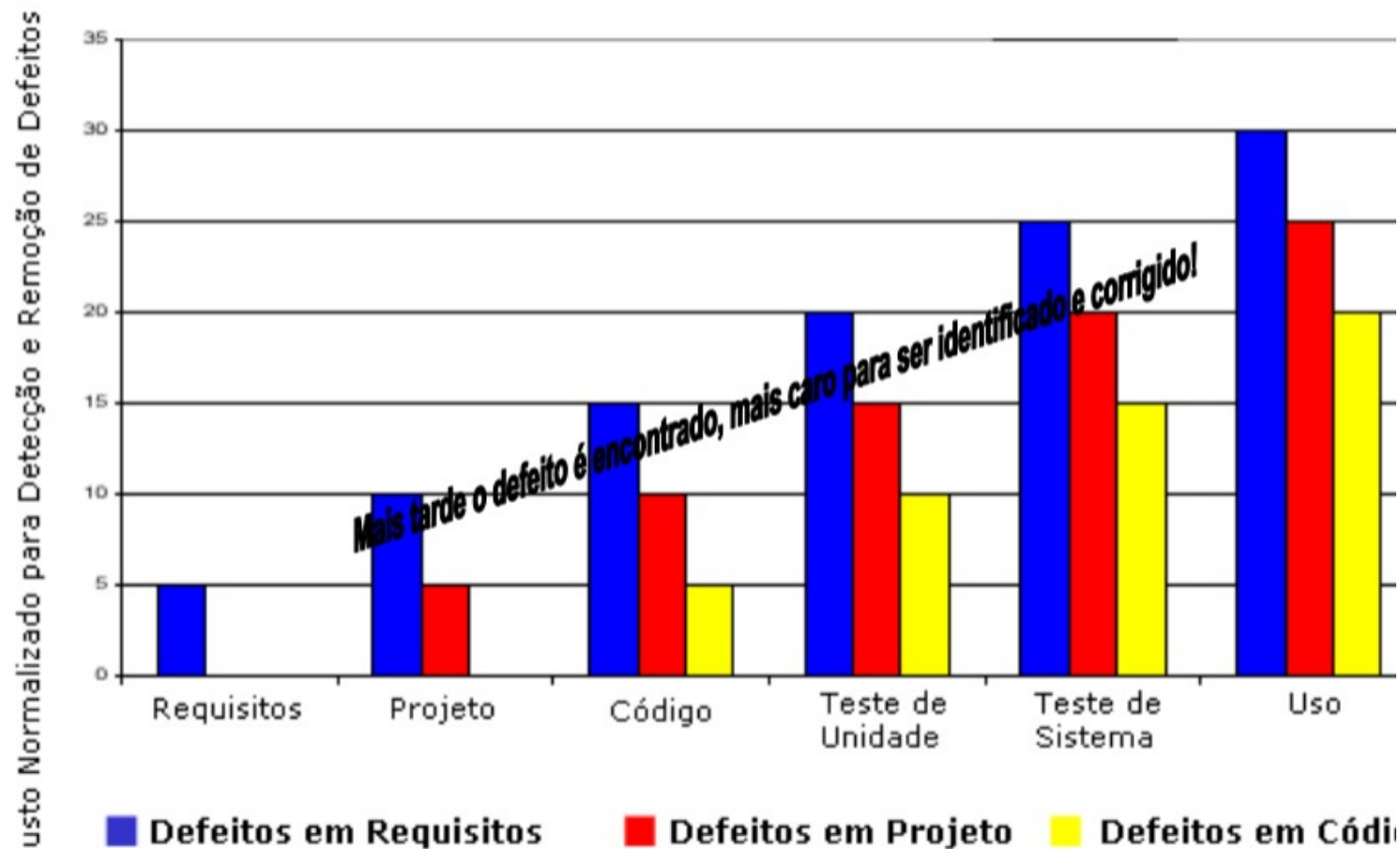
Uma alta taxa de atividades de inspeção ao longo do processo pode representar de 5% a 10% do esforço de desenvolvimento.

Inspeções em requisitos e projeto, conduzidas no JPL
– Nasa Jet Propulsion Laboratory (Miller, 1990);

- Identificação mais cedo de defeitos.



Inspeção



Quando e Como os Testes Entram no Processo de Desenvolvimento de Software

